DBMS LAB 8 — EXCEPTIONS & TRIGGERS

All 8 Programs — Oracle PL/SQL Solutions

Source: Lab 8 programs from the supplied DBMS Lab Manual. Q4/Q5/Q7 include the supporting objects needed by the requested trigger.

1. Demonstrate INVALID_NUMBER, NO_DATA_FOUND and TOO_MANY_ROWS.

```
SET SERVEROUTPUT ON;
DECLARE
 v_empno EMP.EMPNO%TYPE:=&empno; v_deptno EMP.DEPTNO%TYPE:=&deptno;
 v_name EMP.ENAME%TYPE; v_num NUMBER;
BEGIN
 BEGIN
  SELECT ENAME INTO v_name FROM EMP WHERE EMPNO=v_empno;
  v_num:=TO_NUMBER(v_name);
 EXCEPTION WHEN INVALID_NUMBER THEN
  DBMS_OUTPUT.PUT_LINE('INVALID_NUMBER: ENAME cannot be converted to NUMBER.');
 END;
 BEGIN
  SELECT ENAME INTO v_name FROM EMP WHERE EMPNO=v_empno;
  DBMS_OUTPUT.PUT_LINE('Employee: '||v_name);
 EXCEPTION WHEN NO_DATA_FOUND THEN
  DBMS_OUTPUT.PUT_LINE('NO_DATA_FOUND: Employee does not exist.');
 END;
 BEGIN
  SELECT ENAME INTO v_name FROM EMP WHERE DEPTNO=v_deptno;
  DBMS_OUTPUT.PUT_LINE('Employee: '||v_name);
 EXCEPTION
  WHEN TOO_MANY_ROWS THEN DBMS_OUTPUT.PUT_LINE('TOO_MANY_ROWS: Multiple employees found.');
  WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('NO_DATA_FOUND: No employee found.');
 END;
END;
/
```

2. Accept Principal, Interest Rate and Years; calculate interest and validate the three conditions.

```
SET SERVEROUTPUT ON;
DECLARE
 p NUMBER:=&principal; r NUMBER:=&interest_rate; y NUMBER:=&years;
 interest_amt NUMBER;
BEGIN
 IF p<=1000 THEN
  DBMS_OUTPUT.PUT_LINE('Error: Principal must be greater than 1000.');
 ELSIF r<5 THEN
  DBMS_OUTPUT.PUT_LINE('Error: Interest rate must be at least 5%.');
 ELSIF y<1 THEN
  DBMS_OUTPUT.PUT_LINE('Error: Duration must be at least 1 year.');
 ELSE
  interest_amt:=p*r*y/100;
  DBMS_OUTPUT.PUT_LINE('Interest to be paid: '||interest_amt);
 END IF;
END;
/
```

3. Accept employee number, display details, use user-defined exceptions, and update salary below 25000 to 25000.

```
SET SERVEROUTPUT ON;
DECLARE
 e EMP.EMPNO%TYPE:=&empno; n EMP.ENAME%TYPE; s EMP.SAL%TYPE;
 ex_not_found EXCEPTION; ex_high_salary EXCEPTION;
BEGIN
 BEGIN
  SELECT ENAME,SAL INTO n,s FROM EMP WHERE EMPNO=e;
 EXCEPTION WHEN NO_DATA_FOUND THEN RAISE ex_not_found;
 END;
 DBMS_OUTPUT.PUT_LINE('Empno: '||e||' Name: '||n||' Salary: '||s);
 IF s>25000 THEN
  RAISE ex_high_salary;
 ELSIF s<25000 THEN
  UPDATE EMP SET SAL=25000 WHERE EMPNO=e;
  COMMIT;
  DBMS_OUTPUT.PUT_LINE('Salary updated to 25000/-');
 ELSE
  DBMS_OUTPUT.PUT_LINE('Salary is already 25000/-');
 END IF;
EXCEPTION
 WHEN ex_not_found THEN DBMS_OUTPUT.PUT_LINE('Employee does not exist.');
 WHEN ex_high_salary THEN DBMS_OUTPUT.PUT_LINE('Salary is greater than 25000/-.');
END;
/
```

4. Create a trigger to record salary updates in SAL_MOD.

```
CREATE TABLE SAL_MOD(
 EMPNO NUMBER(10), DNAME VARCHAR2(30), OLD_SAL NUMBER(10,2),
 MOD_DATE DATE, MODIFIER VARCHAR2(128)
```

```
);

CREATE OR REPLACE TRIGGER TRG_SAL_MOD
AFTER UPDATE OF SAL ON EMP
FOR EACH ROW
BEGIN
 IF NVL(:OLD.SAL,-1)<>NVL(:NEW.SAL,-1) THEN
  INSERT INTO SAL_MOD(EMPNO,DNAME,OLD_SAL,MOD_DATE,MODIFIER)
  SELECT :OLD.EMPNO,DNAME,:OLD.SAL,SYSDATE,
         SYS_CONTEXT('USERENV','SESSION_USER')
  FROM DEPT WHERE DNO=:OLD.DEPTNO;
 END IF;
END;
/
```

5. Create a trigger for additional PRJ_FUND eligibility based on elapsed project duration.

```
CREATE OR REPLACE TRIGGER TRG_PRJ_FUND_ELIGIBILITY
AFTER UPDATE OF PRJ_FUND ON PROJ
FOR EACH ROW
DECLARE
 duration_days NUMBER; elapsed_days NUMBER;
 additional_fund NUMBER; eligible NUMBER;
BEGIN
 duration_days:=:OLD.END_DATE-:OLD.STRT_DATE;
 elapsed_days:=SYSDATE-:OLD.STRT_DATE;
 additional_fund:=NVL(:NEW.PRJ_FUND,0)-NVL(:OLD.PRJ_FUND,0);

 IF duration_days<=0 THEN
  DBMS_OUTPUT.PUT_LINE('Invalid project duration.');
 ELSIF additional_fund>0 THEN
  IF elapsed_days/duration_days<0.5 THEN
   eligible:=additional_fund*0.70;
  ELSE
   eligible:=additional_fund*0.40;
  END IF;
  DBMS_OUTPUT.PUT_LINE('You are eligible for Rs. '||
                       TO_CHAR(eligible,'9999990.00'));
 END IF;
END;
/
```

6. Create a trigger that prevents an employee's salary from being reduced.

```
CREATE OR REPLACE TRIGGER TRG_PREVENT_SALARY_REDUCTION
BEFORE UPDATE OF SAL ON EMP
FOR EACH ROW
BEGIN
 IF :NEW.SAL<:OLD.SAL THEN
  RAISE_APPLICATION_ERROR(-20001,'Salary reduction is not allowed.');
 END IF;
END;
/
```

7. Create a trigger that logs removed dependents.

```
CREATE TABLE DEPENDENT_DELETE_LOG(
 DEPID NUMBER, EMPNO NUMBER, DEP_NAME VARCHAR2(50), GENDER CHAR(1),
 DATEOFBIRTH DATE, RELATIONSHIP VARCHAR2(30), NOM_PCT NUMBER(5,2),
 MED_COVER VARCHAR2(30), DELETED_DATE DATE, DELETED_BY VARCHAR2(128)
);

CREATE OR REPLACE TRIGGER TRG_DEP_DELETE_LOG
AFTER DELETE ON DEPENDENT
FOR EACH ROW
BEGIN
 INSERT INTO DEPENDENT_DELETE_LOG
 (DEPID,EMPNO,DEP_NAME,GENDER,DATEOFBIRTH,RELATIONSHIP,
  NOM_PCT,MED_COVER,DELETED_DATE,DELETED_BY)
 VALUES
 (:OLD.DEPID,:OLD.EMPNO,:OLD.DEP_NAME,:OLD.GENDER,:OLD.DATEOFBIRTH,
  :OLD.RELATIONSHIP,:OLD.NOM_PCT,:OLD.MED_COVER,SYSDATE,
  SYS_CONTEXT('USERENV','SESSION_USER'));
END;
/
```

8. Create a trigger to prevent salary decrease.

```
CREATE OR REPLACE TRIGGER TRG_NO_SALARY_DECREASE
BEFORE UPDATE OF SAL ON EMP
FOR EACH ROW
WHEN (NEW.SAL<OLD.SAL)
BEGIN
 RAISE_APPLICATION_ERROR(-20002,'Salary cannot be decreased.');
END;
/
```